TeleMed AI — Student MVP Product Requirements Document (PRD)

1. Project Summary

Product Name: TeleMed AI
Product Type: Student MVP for telemedicine and AI medical report summarization

Objective

Build a practical MVP that lets patients and doctors connect through real-time text consultation, while also allowing patients to upload medical PDF reports and receive simple, patient-friendly AI summaries.

Primary Learning Goals

This project is designed for students to learn:

- Full-stack application development
- Role-based authentication and authorization
- Real-time chat and live data syncing
- File upload and secure storage
- AI API integration for document summarization
- Appointment booking and consultation workflows
- Admin dashboard and data management

---

2. Product Vision

TeleMed AI should simulate a lightweight digital clinic where:

- A **patient** can register, complete medical onboarding, upload a PDF report, book a consultation, and chat with a doctor.
- A **doctor** can log in, review appointments, read patient context, see AI-generated report summaries, and chat in real time.
- An **admin** can monitor all users, appointments, and system activity in a read-only dashboard, and can also create doctor accounts.

This MVP is intentionally focused and does not include advanced hospital systems, payments, or video calls.

---

3. Supported Project Variants

Students can build the product in different forms depending on the chosen stack.

A. Web App

Recommended for most students.

Frontend options:

- **Next.js**
- **React.js**

Best for:

- Browser-based patient portal
- Doctor dashboard
- Admin dashboard

B. Mobile App

For students who want a mobile-first experience.

Frontend option:

- **React Native**

Best for:

- Patient mobile app
- Doctor mobile app
- Admin access through a separate web panel or mobile-compatible admin view

C. Backend Options

Students may choose one of the following backend approaches:

1. **Supabase backend**
2. Fastest option for MVP
3. Built-in authentication
4. PostgreSQL database
5. Realtime subscriptions
6. File storage
7. **Node.js backend**
8. Better for students who want custom backend logic
9. Can use Express.js or NestJS

10. Can connect to PostgreSQL, Firebase, or Supabase
11. Suitable for more advanced architecture

Recommended Stack Combinations

- **Web MVP:** Next.js + Supabase
- **Mobile MVP:** React Native + Supabase
- **Advanced Full-Stack MVP:** React / Next.js / React Native + Node.js backend + PostgreSQL

---

4. Target Users and Roles

4.1 Patient

A patient is a user who:

- Creates an account
- Completes profile onboarding
- Uploads medical PDF reports
- Books appointments
- Chats with a doctor in real time

4.2 Doctor

A doctor is a user who:

- Logs in
- Views assigned appointments
- Reviews patient onboarding data
- Reads AI-summarized reports
- Chats with patients in real time

4.3 Admin

An admin is a system user who:

- Views all registered patients
- Views all registered doctors
- Monitors all appointments
- Reviews consultation logs
- Creates doctor accounts
- Manages doctor access
- Cannot edit medical data in the MVP

---

5. Core MVP Features

Epic 1: Authentication and Role-Based Access

Goal

Allow users to sign up and log in as either a patient or a doctor, with admin-controlled doctor onboarding.

Requirements

- Users must be able to create an account using email/password or another approved auth method.
- During sign-up, the user must select a role:
- Patient
- Doctor
- Role must be stored in the database and used for access control.
- Users should only see the pages allowed for their role.

Access Rules

- Patients should access patient pages only.
- Doctors should access doctor pages only.
- Admin should access admin pages only.
- Unauthorized users must be redirected away from restricted pages.

Doctor Account Creation Flow

- Doctor accounts can be created by the **admin** from the admin dashboard.
- When the admin creates a doctor account, the system must generate either:
- a **temporary auto-generated password**, or
- a **secure invite link** for the doctor to activate the account.
- If a temporary password is used, the doctor should be required to change it on first login.
- If an invite link is used, the doctor should complete account activation through the link before logging in.
- Admin-created doctor accounts must automatically receive the **Doctor** role.

Epic 2: Mandatory Patient Onboarding

Goal

Collect important medical context before the patient can use the system.

Requirements

On first login, every patient must complete a profile form before accessing the dashboard.

Onboarding Fields

Required fields:

- Full Name
- Age
- Gender
- Weight
- Height
- Allergies
- Current Medications
- Pre-existing Conditions
- Emergency Contact Number

Optional fields:

- Blood Group
- Notes
- Past Surgeries
- Smoking Status
- Chronic Illness Notes

Validation Rules

- Patient cannot skip onboarding.
- Patient cannot book appointments until onboarding is saved.
- Patient cannot access report upload or consultation features until profile completion is finished.
- All required fields must be validated before submission.

UX Behavior

- After first login, redirect patient to onboarding form.
- After saving, redirect to patient dashboard.
- If onboarding is incomplete, keep redirecting to onboarding until completed.

Epic 3: My Reports — PDF Upload and AI Summary

Goal

Allow patients to upload medical PDFs and get simplified AI summaries.

Requirements

- Patients must have a dedicated **My Reports** section.
- File input must accept **PDF only**.
- Uploading images, Word files, or other formats is not allowed in the MVP.
- After upload, the PDF must be stored securely in file storage.

• The system must send the PDF content or extracted text to the AI model for summarization.
• The AI summary must be saved in the database along with the file reference.
• The patient must see:
• Report name
• Upload date
• AI-generated summary
• Download/view link if available

AI Summarization Flow

1. Patient uploads PDF
2. File is stored in storage
3. PDF text is extracted or processed
4. AI model generates simple summary
5. Summary is saved in database
6. Summary is displayed to patient

Suggested AI Prompt

“Act as a medical assistant. Explain this medical report in simple, easy-to-understand language for a patient. Avoid complex terminology and summarize key findings, possible concerns, and general meaning.”

UI Requirements

• List all uploaded reports
• Show current processing status:
• Uploaded
• Processing
• Completed
• Failed
• Display summary below the uploaded report

Notes

• The AI summary is informational only and does not replace a doctor’s opinion.
• The system should include a disclaimer that the AI summary is not medical advice.

Epic 4: Appointment Booking

Goal

Let patients book a consultation with a doctor.

Requirements

• Patients can view a list of active doctors.
• Patients can select a doctor.

- Patients can choose an available time slot.
- The booking creates an appointment record in the database.
- Appointment must link:
- Patient ID
- Doctor ID
- Date/time
- Status
- Consultation room reference

Appointment Statuses

- Pending
- Confirmed
- Completed
- Cancelled

MVP Scheduling Rules

To keep scope simple:

- Doctors are considered available for the MVP.
- No advanced calendar conflict engine is required.
- No recurring schedules are required.
- No rescheduling flow is required unless added by the team.

Patient Booking Rules

- Patients must have a completed profile before booking.
- Patients can only book for themselves.
- Booking confirmation should be visible immediately after creation.

Epic 5: Live Consultation Room

Goal

Provide a real-time consultation experience with contextual patient information.

Layout

The consultation room must use a split-screen layout.

Left Panel: Context

Display:

- Patient onboarding data

- Allergies
- Medications
- Pre-existing conditions
- Uploaded reports
- AI-generated report summaries

Right Panel: Chat

Display:

- Real-time text chat
- Appointment-specific messages only
- Message history for the current consultation

Chat Requirements

- Chat must use real-time syncing.
- Messages should appear instantly for doctor and patient.
- Messages must be tied to a specific `appointment_id`.
- Users should not see chats from other appointments.
- Page refresh should not break chat history.
- Messages should include:
- Sender
- Timestamp
- Message content

Chat Status

Optional status indicators may include:

- Doctor joined
- Patient joined
- Consultation in progress
- Consultation ended

Constraints

- Text chat only
- No voice call
- No video call
- No attachments in chat for MVP

Epic 6: Doctor Dashboard

Goal

Allow doctors to manage consultations for the day.

Requirements

- Doctor can log in and see a list of their appointments.
- Doctor dashboard should show:
- Patient name
- Appointment time
- Status
- Quick access to consultation room
- Doctor can open one appointment and see patient context in split-screen mode.
- Doctor can chat in real time with the patient.

Doctor Workflow

1. Log in
2. View today’s appointments
3. Open consultation
4. Review patient history and AI summary
5. Chat with the patient
6. Mark consultation complete

Epic 7: Admin Overview Dashboard

Goal

Give admins a simple, read-only system overview and doctor onboarding control.

Requirements

Admin dashboard must include three tabs:

Tab 1: Patients

Show a table of all registered patients with:

- Name
- Email
- Age
- Gender
- Status
- Registration date

Tab 2: Doctors

Show a table of all registered doctors with:

- Name

- Email
- Specialty if available
- Status
- Registration date
- Account creation method
- Invite sent / password reset status

Tab 3: Appointments / System Log

Show all appointments with:

- Patient name
- Doctor name
- Date/time
- Status
- Consultation ID or appointment ID

Admin Doctor Management

- Admin can create a new doctor account from the dashboard.
- Admin enters doctor details such as:
- Name
- Email
- Specialty
- Status
- The system generates either:
- a temporary auto-generated password, or
- a secure invite link
- The created login details are shared with the doctor so the doctor can activate and access the account.
- Newly created doctor accounts should appear in the Doctors tab automatically.

Admin Restrictions

- Read-only dashboard for system data
- No editing of patient medical records
- No deleting of consultations in MVP
- No financial data
- No advanced reporting required

---

6. Page / Screen List

Public Screens

- Landing page
- Sign up page
- Login page

Patient Screens

- Patient onboarding form
- Patient dashboard
- My Reports page
- Appointment booking page
- Consultation room
- Profile page

Doctor Screens

- Doctor dashboard
- Appointment detail page
- Consultation room

Admin Screens

- Admin dashboard
- Patients tab
- Doctors tab
- Appointments tab
- Create Doctor screen / modal

---

7. Data Models

User

- id
- full_name
- email
- role (`patient`, `doctor`, `admin`)
- account_status
- created_at
- updated_at

Patient Profile

- id
- user_id
- age
- gender
- weight
- height
- allergies
- current_medications
- pre_existing_conditions

- emergency_contact
- completed_onboarding
- created_at
- updated_at

Doctor Profile

- id
- user_id
- specialty
- bio
- active_status
- created_by_admin
- invite_token
- temp_password_status
- created_at
- updated_at

Medical Report

- id
- patient_id
- file_name
- file_url
- file_type
- ai_summary
- upload_status
- created_at
- updated_at

Appointment

- id
- patient_id
- doctor_id
- scheduled_time
- status
- created_at
- updated_at

Chat Message

- id
- appointment_id
- sender_id
- message
- created_at

8. Recommended Technical Architecture

Option A: Supabase-First MVP

Best for fast student delivery.

Includes

- Authentication
- PostgreSQL database
- Realtime chat
- Storage for PDF files
- Row-level security

Advantages

- Faster development
- Less backend setup
- Easier for MVP
- Good for real-time workflows

Option B: Node.js Backend

Best for students who want custom backend logic.

Includes

- Node.js API
- PostgreSQL or Supabase database
- Custom endpoints for auth, uploads, appointments, chat, and admin doctor creation
- Optional socket or realtime layer

Advantages

- More control
- Better for advanced system design
- Useful for students learning backend engineering deeply

---

9. Non-Functional Requirements

Performance

- Pages should load quickly
- Chat messages should sync in near real time

- PDF upload and AI processing should show progress status

Security

- Role-based access control required
- Patient medical data must be protected
- PDF files must be stored securely
- Only authorized users can read consultation data
- Doctor invite links or temporary passwords must be secure and time-limited where applicable

Reliability

- If AI processing fails, the report should remain saved with a failed status
- The system should handle reconnects without losing chat history

Usability

- Clean and simple UI
- Easy navigation for non-technical patients
- Doctor dashboard should be efficient for daily use

---

10. Out of Scope for MVP

The following are strictly excluded from the MVP:

- Video calling
- Audio calling
- Payments
- Insurance claims
- Pharmacy ordering
- Prescription generation
- Advanced doctor scheduling
- Calendar sync with external systems
- Email verification flow
- Password reset flow
- Word / DOCX / image uploads
- Multi-specialty routing logic
- Advanced analytics
- Push notifications
- Mobile offline mode

---

11. Acceptance Criteria

The project is complete when:

- Users can sign up as patient or doctor
- Admin can create doctor accounts
- Doctor accounts can be delivered using a temporary password or invite link
- Patients must complete onboarding before using the system
- Patients can upload PDF reports
- AI summary is generated and displayed
- Patients can book an appointment with a doctor
- Doctor can see appointments and patient context
- Real-time chat works for a specific appointment
- Admin can view all patients, doctors, and appointments in read-only mode
- The app works reliably in the chosen frontend and backend stack

---

12. Student Deliverables

Students should submit:

- Source code
- Database schema
- Setup instructions
- Environment variable list
- Feature walkthrough
- Screen flow documentation
- Final demo video or screenshots
- Short explanation of which stack they selected and why

---

13. Suggested Team Decision Rules

Students may choose based on their goals:

Choose Next.js

If they want:

- Web app
- Better dashboard experience
- Server-side rendering
- Faster full-stack web delivery

Choose React.js

If they want:

- Pure frontend web app
- Separate API backend
- Simpler app structure for learning

Choose React Native

If they want:

- Mobile app
- Android/iOS-friendly experience
- Patient-first app delivery

Choose Supabase

If they want:

- Faster MVP
- Built-in auth, storage, realtime
- Less backend boilerplate

Choose Node.js Backend

If they want:

- Custom APIs
- Deeper backend learning
- More control over business logic

---

14. Final Product Definition

TeleMed AI MVP is a role-based telemedicine system that supports:

- Patient onboarding
- PDF upload and AI report summarization
- Doctor appointment booking
- Real-time consultation chat
- Admin visibility into system data
- Admin-created doctor accounts with secure access delivery

The purpose of this product is not to create a full production healthcare platform, but to give students a strong, realistic, end-to-end software engineering project.

15. Recommended Default Stack for Students

If students are unsure, the best default choice is:

- **Frontend:** Next.js
- **Backend:** Supabase
- **Database:** PostgreSQL
- **Realtime:** Supabase Realtime
- **Storage:** Supabase Storage
- **AI:** Gemini 1.5 Flash API
- **Deployment:** Vercel for frontend, Supabase hosted backend

This combination keeps the project manageable while still teaching real-world product development.